

Прогнозне обслуговування без міток

Як модель вчиться визначати, що «потрібне обслуговування», зі звичайної телеметрії — і продовжує навчатися від операторів під час роботи

Повний огляд моделі прогнозного обслуговування без міток, побудованої як нейронна мережа Jneopallium: що робить кожен нейрон, чому не потрібна історія відмов, як вона вдосконалюється зі зворотного зв'язку операторів без переустановлення та чому залишається суто рекомендаційною.

Документ	Архітектурна стаття
Продукт	Jneopallium Self-Supervised Maintenance Guardian
Модуль	self-supervised-maintenance 1.0.0-label-free-continuous
Середовище виконання	Java-нейрони · net.neuron.impl.ssmaint
Навчання	Python (лише початкове припасування) · стандартна бібліотека
Безпека	ADVISORY / лише читання — ніколи не керує
Автор	Дмитро Раковський — Харків, Україна
Дата	2 липня 2026
Ліцензія	BSD 3-Clause

Зміст

1. Проблема в одному абзаці
2. Для нефахівця: сама ідея
3. Що спостерігає модель
4. П'ять шарів коротко
5. Шар 1 — самокероване відновлення
6. Шар 2 — гіпотеза обслуговування
7. Шар 3 — безперервне навчання зі зворотного зв'язку
8. Шар 4 — рекомендаційний шлюз лише для читання
9. Сигнали, що все поєднують
10. Чому не потрібні мітки
11. Як воно навчається без переустановлення
12. Модель безпеки
13. Чого воно не може (чесні межі)
14. Форма розгортання

1 Проблема в одному абзаці

Більшість моделей прогнозного обслуговування є керованими: їм подають історію минулих відмов і ремонтів, і вони вчаться розпізнавати наростання перед кожною. Це працює — коли у вас є розмічена історія відмов. У більшості майданчиків її немає. Є роки телеметрії та журнал обслуговування, що уривчастий, у вільній формі або відсутній. Self-Supervised Maintenance Guardian — це модель саме для такої ситуації: вона вчиться, який вигляд має «здоровий» стан, безпосередньо з телеметрії, позначає відхилення як несправності, що розвиваються, із запасом часу, і згодом уточнює себе завдяки єдиному виду нагляду, що надходить безкоштовно — оператор каже «так, це справжнє» або «ні, це хибна тривога».

2 Для нефахівця: сама ідея

Уявіть досвідченого оператора, який роками слухає насос. Ніхто не давав йому підручник розмічених відмов; він просто знає, як машина звучить у добрий день, і помічає, коли щось відхиляється. Тоді він не кричить «ВІДМОВА» — він каже «пильнуй підшипник 2, він гріється». Якщо він кілька разів помилився, він переналаштовує власне відчуття «норми». Ця модель — той оператор, записаний у код.

- Вона вчиться «нормі» із самих даних — розмічені відмови не потрібні.
- Вона дивиться на кілька сенсорів разом, бо справжня несправність проявляється як закономірність між ними, а не одна стрілка в червоній зоні.
- Вона піднімає прапорець лише тоді, коли відхилення стійке, трендове й узгоджене — а не на миттєвий сплеск.
- Вона підказує, на який тип проблеми це схоже і приблизно скільки у вас часу.
- Коли оператор підтверджує чи відхиляє прапорець, вона тихо коригується — не зупиняючись.
- Вона ніколи не торкається машини. Вона лише радить людині.

Одне речення для запам'ятовування

Цілі моделі — це самі сенсори, а не мітка відмови, тому її можна навчати лише на звичайних робочих даних.

3 Що спостерігає модель

Єдиний вхід — це кадр телеметрії: одне зчитування кожного сенсора в один момент, позначене поточним режимом роботи (низьке / середнє / високе навантаження). Еталонна збірка використовує вісім пов'язаних каналів насосної установки:

Канал	Що це	Чому важливо
suction_pressure	Тиск на вході насоса	Падає при кавітації
flow	Витрата процесу	Задає робочу точку; нестабільна при кавітації
pump_speed	Швидкість валу	Нормалізує решту
pump_power	Споживана потужність	Зростає з зносом і втратою ефективності

vibration_rms	Рівень вібрації	Зростає при зносі підшипника й кавітації
bearing_temp	Температура підшипника	Зростає при зносі підшипника
process_temp	Температура процесу	Дрейфує при відмові її сенсора
valve_position	Хід клапана	Коливається при проблемах контуру/залипанні

Канали фізично пов'язані — потужність слідує за витратою, температура підшипника за вібрацією — і саме цей зв'язок модель використовує. Позначка режиму дозволяє оцінювати зчитування відносно навантаження, тож напружена зміна не виглядає як несправність.

4 П'ять шарів коротко

Шар	Нейрон	Перетворює ... на ...
0	(прийом телеметрії)	кадр сенсорів → AssetTelemetrySignal
1	CrossSensorReconstructionNeuron	телеметрія → залишок відновлення
2	MaintenanceHypothesisNeuron	залишок → гіпотеза + запас часу
3	FeedbackAdaptationNeuron	зворотний зв'язок → оновлення порога
4	SsAdvisoryGateNeuron	гіпотеза (+ оновлення) → рекомендація

Шари 1–2 і 4 утворюють швидкий шлях виявлення (телеметрія на вході, рекомендація на виході). Шар 3 — повільний шлях навчання (зворотний зв'язок на вході, пороги на виході), що живить шар 4. Кожен нейрон однопризначений; потужність додається шарами, а не заплутаною логікою в одному нейроні.

5 Шар 1 — самокероване відновлення

Це серце ідеї без міток. Для кожного сенсора нейрон містить невелику лінійну модель — припасовану офлайн на довіреному здоровому вікні — яка передбачає цей сенсор за всіма іншими після стандартизації за режимом. Під час роботи вона порівнює передбачення з реальністю:

```
residual_i = actual_i - predicted_from_peers_i    (стандартизовано)
total      = середнє по сенсорах від residual_i2
```

Великий **total** означає, що спільна картина більше не виглядає здоровою. Великий **окремий** залишок означає, що один сенсор суперечить усім іншим — класична ознака сенсора, що виходить з ладу. Оскільки ціллю кожної моделі є інший сенсор, мітка відмови не потрібна; дані навчають самі себе. Нейрон також повідомляє частку **зсуву домену**: скільки сенсорів лежить далеко поза межами, на яких він навчався, тобто наскільки незвичне це зчитування.

Простими словами

Нейрон запитує кожен сенсор: «зважаючи на те, що роблять твої сусіди, яким має бути твоє показання?» Коли реальне значення сенсора відхиляється від того, що передбачають сусіди, цей розрив і є раннім попередженням.

6 Шар 2 — гіпотеза обслуговування

Один залишок — це шум; випадок обслуговування — це історія в часі. Нейрон гіпотези тримає для кожного активу чотири поточні величини й поєднує їх:

- **Тренд** — повільне ковзне середнє залишку та його нахил, щоб відрізнити стійке зростання від сплеску.
- **Точка зміни** — детектор Пейджа-Хінклі, що спрацьовує, коли поведінка переходить на новий рівень.
- **Тяжкість** — рівень у власних перцентильних одиницях активу, тож «1.0» означає «на 99-му перцентилі, який цей актив колись показував». Каліброване зі здорової історії, не з міток.
- **Свідчення** — накопичувач, що зростає лише коли відхилення тяжке, трендове, узгоджене щонайменше по двох сенсорах і не є просто зсувом домену.

Він приписує **сімейство несправності** евристично за тим, які сенсори домінують у залишку (підшипник, кавітація, сенсор, енергія, коливання чи чесне «невідоме»), і екстраполює тренд до базової межі, щоб оцінити **запас часу** — заміник без міток для «скільки лишилось до потреби в увазі». Невизначеність зростає зі зсувом домену, тож незнайомий актив читається як «спостерігаю, не впевнений» замість впевненої хибної відповіді.

7 Шар 3 — безперервне навчання зі зворотного зв'язку

Коли оператор позначає рекомендацію як підтверджену чи хибну, цей вердикт надходить як `OperatorFeedbackSignal`. Нейрон адаптації тримає для кожного сімейства зсув поверх статичного базового порога й рухає його:

- **Хибне спрацювання** підвищує поріг цього сімейства — менше зайвих тривог такого типу.
- **Підтверджена потреба** трохи послаблює його — повнота вже висока, тож поштовх малий і навмисно асиметричний.

Кожне оновлення **обмежене** (зсув затиснуто), з **обмеженням частоти** (мінімальний інтервал між змінами) і **заморожене під час зсуву домену**, щоб новий чи аномальний період не отруїв пороги. Нейрон володіє цим станом, який середовище зберігає у сховищі — тож навчання переживає перезапуск. Головне: ніщо з цього не потребує перезбирання чи переустановлення моделі: він видає `ThresholdUpdateSignal`, який шлюз застосовує на місці.

8 Шар 4 — рекомендаційний шлюз лише для читання

Кінцевий етап перетворює гіпотезу на рекомендацію для людини, коли накопичене свідчення долає живий поріг сімейства, з урахуванням вікна дедуплікації для кожного активу, щоб одна несправність, що розвивається, не повідомлялася щотакту. Він тримає пороги в живій мапі, яку шлях зворотного зв'язку змінює на місці — саме так вердикти операторів переформовують поведінку шлюзу під час роботи процесу. Єдине, що він може видати — це `MaintenanceAdvisorySignal`; шляху до виконавчого механізму в нього немає за побудовою.

Вартий уваги захист від регресії

Вікно дедуплікації та обмеження частоти зворотного зв'язку спершу використовували сентинел, що переповнювався на реальних мітках часу в мілісекундах, що придушило б найпершу рекомендацію та перший зворотний зв'язок у продакшені. Обидва виправлено й покрито тестами.

9 Сигнали, що все поєднують

Сигнал	Несе	Каданс (епоха/цикл)
AssetTelemetrySignal	один багатосенсорний кадр + режим	1 / 1 (швидкий)
ReconResidualSignal	total + залишки по сенсорах + зсув домену	1 / 1
HealthHypothesisSignal	сімейство, тяжкість, свідчення, запас часу, невизначеність	2 / 2
OperatorFeedbackSignal	підтверджено / хибне + походження	10 / 2 (повільний)
ThresholdUpdateSignal	поріг сімейства + зсув	10 / 2
MaintenanceAdvisorySignal	рекомендація лише для читання + запас часу	2 / 2

Кожен сигнал — типізований ISignal зі своєю ProcessingFrequency, тож швидкий шлях виявлення й повільний шлях навчання плануються незалежно на одному рушії.

10 Чому не потрібні мітки

Кожна ціль навчання є самокерованою — її ціль це те, що модель може перевірити з реальністю без людини: відновити сенсор за сусідами, калібрувати тяжкість за власною історією активу, екстраполювати тренд до межі. Жодна з них не вимагає знати, що конкретна минула подія «була відмовою підшипника». Наслідок практичний: можна розгортати на майданчику, де є телеметрія, але немає розміченої історії обслуговування, а це більшість майданчиків.

11 Як воно навчається без переустановлення

Навчання під час роботи живе у стані нейронів, а не в упакованому JAR. Робітник працює безперервно; нейрон адаптації коригує пороги на повільному циклі й зберігає їх у налаштованому сховищі. Окрема дисципліна «чемпіон/претендент» — рекомендований спосіб безпечно розвивати важчі параметри: стабільний чемпіон видає рекомендації, поки претендент адаптується в тіні й просувається лише коли перевершує чемпіона за незалежними наслідками. Ніщо тут не перериває робочий процес.

12 Модель безпеки

- **Інваріант «лише рекомендація».** SsMaintConfig забороняє вимикати рекомендаційний режим; жоден нейрон модуля не має шляху до виконавчого механізму.
- **Шар безпеки ніколи не навчається.** Адаптація порогів торкається оцінок і порогів тривоги — ніколи жорсткого блокування чи шлюзу безпеки.
- **Чесна невизначеність.** Зсув домену й невизначеність — повноправні виходи, тож модель може відмовитися заявляти впевненість щодо незнайомого активу.
- **Захист від отруєння.** Адаптація заморожується під час зсуву домену; базові рівні адаптуються лише з довірених здорових періодів.

13 Чого воно не може (чесні межі)

Режим без міток виявляє **деградацію**, а не договірно належний ремонт. Сімейства несправностей — евристичні гіпотези, доки не накопичаться підтверджені події, тож очікуйте більше хибних спрацювань, ніж у повністю навченої на мітках моделі — саме для цього існує цикл зворотного зв'язку. Також потрібне **переважно здорове** вікно навчання (або однолітки з парку), щоб визначити норму; цілком новий актив без історії починає у режимі високої невизначеності з усвідомленням зсуву домену, а не робить упевнені заяви.

14 Форма розгортання

Навчена модель — це збірка Jneopallium: дескриптор моделі, IContext і poshарова конфігурація нейронів із припасованими параметрами — завантажується стандартним запускателем Entry в режимі local, HTTP чи gRPC. Телеметрія надходить через звичні входи OPC UA / MQTT; рекомендації виходять через агрегатор виходу (JSONL, Kafka чи MQTT). Супутній посібник з розгортання охоплює повний регламент; посібник з навчання охоплює припасування параметрів з вашого історіана.

Self-Supervised Maintenance Guardian — виявлення без міток, навчання зі зворотного зв'язку та режим безпеки, що ніколи не виходить за межі рекомендації. Це модель для майданчика, де є дані, але ще немає розміченої історії відмов.